GLOBAL · ADOPTION ETHOS

The Trust Framework

You don't have to trust it. You have to be able to check it. Eight practices that make AI-assisted work defensible — plus why **Build vs Buy** is about to be your team's greatest superpower.

For: **anyone being asked to adopt AI or automation who would like to do it without becoming the person holding the bag.**

Cost: **nothing. This is a set of practices, not a product. Every one can be implemented with tools your organisation already owns.**

You don't have to trust it. You have to be able to check it.

If AI in your workplace makes you uneasy, you are probably not being irrational. You have most likely watched a system change in a way nobody could fully explain, seen the blame settle on whoever was standing nearest, and noticed that "the tool did it" is not a defence anyone accepts.

That instinct is worth keeping. The mistake is thinking the choice is between adopting and refusing.

There is a third position, and it is the strong one: **adopt it, and require that it produce a record.** You do not need to understand how a model works to insist that anything acting on your organisation's behalf leaves behind what changed, when, and under whose authority. That is not a technical skill. It is a standard — and holding a standard is something you are already qualified to do.

What follows is that standard, in eight parts. None of them require a purchase. All of them are things you can ask for, specify, or check.

1. Record as it happens. Never reconstruct.

The single most expensive mistake is deciding to work out what happened afterwards. Reconstruction relies on exports, logs kept for other reasons, and people's memory — and it gets weaker every week, precisely as the question becomes more serious.

The practice: when something consequential happens, write down what happened *at that moment*, including why it was permitted. A row created at the time is evidence. The same row assembled six months later is a reconstruction, and everyone in the room will know the difference.

How to check it: ask for something that happened last quarter. If the answer takes more than a few minutes to assemble, you don't have a record — you have an archaeology project.

2. One identity per actor. Never a shared login.

Shared credentials — a service account several systems use, a password a team passes around, an API key three automations share — feel efficient. They quietly destroy your ability to answer the only question that matters after an incident: *which one did this?*

This is not fixable later. If two things used the same credential, no amount of analysis separates them, because the information was never captured.

The practice: every actor — person, system, or AI agent — gets its own identity. Yes, it is more setup. It is also the difference between switching one thing off and switching everything off.

How to check it: ask whether one automation can be revoked without disturbing the others. If not, they're sharing an identity.

3. Authority is bounded, and it expires.

Access is usually granted as a state: you have it, or you don't, indefinitely. That was tolerable when the actors were people who left visible traces. It is not tolerable for software that acts thousands of times an hour.

The practice: grant authority with limits attached — what it covers, how much it may spend or change, and when it stops. An agent should hold permission to do a specific bounded thing, not a credential that lets it do anything the credential could do.

How to check it: ask what an automation is *not* allowed to do, and how that limit is enforced. If the answer describes what it was configured to do rather than what would stop it, there is no boundary — only an intention.

4. Consent carries its context.

Consent recorded as yes-or-no is not evidence. When a regulator, a customer or a lawyer asks, the questions are: *when* was it given, by *what* mechanism, under *which* version of your terms, and for *what* purpose.

The practice: store those four things with every consent event, and never overwrite them. A change adds a new record; it doesn't edit the old one.

How to check it: pick one customer. Ask to see their consent as it stood on a specific date last year. A true/false field cannot answer that question.

5. Anything you might be asked about later is a series, not a value.

Systems store what things are now. Prices, permissions, settings, statuses — one current value, overwritten on change. But almost every serious question is retrospective: what was the lowest price in the thirty days before that promotion; who had access in March; what did the policy say when the incident happened.

The practice: for anything a regulator, auditor, customer or court could ask about, keep the history rather than the latest state.

How to check it: ask what a given value was three months ago, and how you'd prove it.

6. Corrections add. They never edit.

The instinct on finding a wrong record is to fix it. That instinct destroys the record's value — because if entries can be edited, none of them prove anything, including the correct ones.

The practice: append-only. A mistake is corrected by adding an entry that says so, with a time and an author. The original stays visible.

How to check it: ask whether a record can be edited after the fact, and by whom. If the answer is "an administrator can," your history is a claim, not evidence.

7. An outsider must be able to check it.

A record only you can read is a record only you vouch for. The moment there's a dispute, that is exactly the wrong position — you are asking the other party to trust the account of the interested party.

The practice: produce records a third party can verify independently. Cryptographic signatures are the strong version. But even simple measures help enormously: timestamps from a source you don't control, records held where you cannot silently alter them, or copies delivered to the counterparty as events occur.

How to check it: ask how a sceptical outsider would confirm a record is genuine without taking your word for it.

8. The record protects the person, not only the organisation.

This is the part usually left unsaid, and for many people it is the real reason to care.

When something built on undocumented decisions fails, it fails architecturally — a constraint nobody wrote down, a permission model that couldn't answer, a migration that was somebody else's schedule. But the person closest to the failure is rarely the person who designed it. It is whoever inherited it.

Without a record, nothing distinguishes a failure you inherited from one you caused. No evidence that the constraint predated you, that you raised the risk, that the fix was outside your authority. The absence doesn't just slow the organisation down — it silently moves the blame to the person least able to have prevented it.

The practice: write down what you were handed, what you flagged, and what you were permitted to change. Not defensively — accurately. It is the same record that answers a regulator and the one that answers *was this reasonable, given what they were given.*

Build vs Buy is about to be your team's greatest superpower

Every practice above runs into the same question: *do we buy something that does this, or build it?*

For twenty years that question had a standing answer. Buy. Building meant a team, a year, and maintenance forever; buying meant a contract and a go-live date. The answer was so reliable that most organisations stopped asking and simply ran a procurement process.

That calculus has changed, and only on one side. AI has collapsed the cost of building the specific thing — the part that encodes how *your* organisation works. It has not collapsed the cost of buying the wrong general thing. If anything it raised it, because a platform you cannot modify is now slower than a team that can.

So the question is live again, and the teams that answer it well will out-perform the teams that outsource the answer. Not because building is virtuous — most of what you run should still be bought — but because knowing which is which is now a decisive capability rather than a procurement formality.

The line worth drawing

Buy the substrate. Databases, edge compute, payment rails, identity providers, certified hosting. These are commodities with real economies of scale,

someone else's compliance burden, and no competitive value in owning. Buying here is unambiguously correct, and the certifications you inherit are genuinely load-bearing.

Build the part that encodes your policy. Which markets you operate in, what consent means in each, who may authorise what, what you must be able to prove and to whom. No vendor can know these, because they are not general facts about commerce — they are specific facts about you, and in many cases about your contracts and your regulators.

The failure mode is buying a platform to satisfy an obligation you have not yet articulated. You end up with a configuration nobody can defend, a dependency nobody can remove, and — the expensive part — an answer to the auditor that is really the vendor's answer, given on your behalf, about facts the vendor never had.

Like Dorothy, you already have what you need

The objection to building is almost always assumed rather than checked: *we don't have the infrastructure.* In most organisations running modern commerce, that is no longer true — the components a buy pipeline would have gone out and procured are already in the building, or free on signup.

- **Source control, review and history** — GitHub, already present in any organisation shipping software, and the substrate for every claim about who changed what.
- **The application and data layer** — a managed backend gives you Postgres, Redis, containerised functions on orchestration you don't administer, and generated OpenAPI/Swagger documentation, available minutes after signup on a free tier and SOC 2 certified at the service levels above it. That is the same stack shape a platform vendor would sell you as a platform.
- **The edge** — Cloudflare, already fronting most of these estates, with native cryptographic primitives, key storage and global execution included rather

than licensed.

- **The commerce system itself** — if you run Shopify, its APIs, webhooks and identity are already yours to build against.

The database point deserves emphasis, because it is where the policy layer actually lives: a Postgres instance whose tables, relationships and access rules **you** design. Not a vendor's schema you configure around, but your definitions — your consent shape, your market registry, your permission model — with security expressed as rules you wrote and can show someone.

And none of it is sitting in a procurement queue with an approval date six months out. That is the part worth internalising. The reason "build" used to lose was never really cost; it was time-to-permission. That constraint is gone for this class of tooling. You can have the substrate running this afternoon, prove the shape against real calls, and present it as a working specification — at which point the procurement conversation is about scale and support rather than about whether the idea works.

Why this is a superpower rather than a preference

Three things follow from getting the line right, and they compound:

You can answer questions in hours instead of quarters. When the policy layer is yours, a regulator's question is a query. When it's the vendor's, it is a support ticket, an export, and a reconstruction — and the answer arrives after the deadline.

Your obligations stop being someone else's roadmap. Every organisation that has waited on a vendor to support a jurisdiction, a consent shape or a data residency requirement knows this cost. It is not a line item; it is an inability to operate, priced as a delay.

You keep the option to change your mind. A bought system that holds your policy holds you. A built policy layer over bought substrate lets you replace the

substrate — which is the only kind of leverage that survives a renewal negotiation.

The honest version

This argues against selling you something, which is the point. If the practices in this document are implemented on a platform you already own, that is a complete success — the framework did its job. The value was never in the tooling; it is in a team that can tell the difference between infrastructure worth renting and judgement worth keeping.

There is a harder truth underneath, and it is worth saying plainly: an organisation whose entire delivery model is reselling bought software has a structural reason not to help you draw this line. That is not bad faith. It is just an incentive, and you should price it in the same way you would price any other supplier's interest in the outcome.

Adopting this without buying anything

Every practice above is a decision, not a product:

- Ask the four consent questions of your current platform and write down the answers.
- Give each automation its own credential. It is an afternoon.
- Put an expiry on access that currently has none.
- Stop overwriting the values you might be asked about; keep the series.
- Make one important log append-only.
- Send a counterparty a copy of the records that concern them, as they happen.
- Write down what you inherited, before you change it.

None of that requires a vendor, a budget line, or an engineering team. It requires someone to decide it matters — which is the actual scarce resource.

Where AI fits

The reason this is urgent rather than merely sensible: self-improving automation changes systems faster than any review cadence. A loop that writes, tests and revises on its own is the largest productivity change most teams will see this decade — and it produces change faster than humans can attribute it.

That is not an argument against adopting it. It is an argument for adopting it **with a record**, because a fast system that can explain itself is an asset, and a fast system that cannot is a liability compounding at speed.

It also resolves the fear honestly. You do not have to become an expert in a technology you distrust. You have to insist that it accounts for itself — and that is a position you can hold on day one, without knowing anything about how it works.

Asking how it is governed is not resistance

There is a version of the AI conversation where anyone raising provenance, consent or authority is filed under obstruction — the person slowing things down. It is worth naming why that reading is backwards.

Governed AI is the version that gets approved. Unbounded agents on shared credentials do not clear a security review, cannot be deployed against regulated data, and stall at exactly the point where they would have started mattering. The person who can say what a safe AI workflow requires is not blocking

adoption; they are describing the only shape of it that survives contact with the organisation.

And there is a plainer reason to engage, which most people feel and few say out loud. Organisations moving to AI workflows end up assessing who can operate them. That assessment is rarely formal and it is almost never about who can build a model — it is about who can specify what the system must do, what it must never do, and how anyone would know. Those are the eight practices above. They are also, not coincidentally, the questions someone with operational judgement is better placed to answer than someone with only engineering skill.

So the transition is not a threat to be endured. It is the moment when knowing how your business actually works — which markets, which obligations, which promises to which customers — becomes the scarce input rather than the assumed background. The people who show up with that, and with a standard for how automation must account for itself, are not made redundant by this shift. They are what makes it possible.

This framework is deliberately vendor-neutral and free to adopt, adapt, or republish with attribution. If you want to see one implementation of it — signed records, bounded agent mandates, publicly verifiable certificates — the technical brief describes how it is built and the architecture comparison explains the trade-offs. Neither is required to use anything above.
